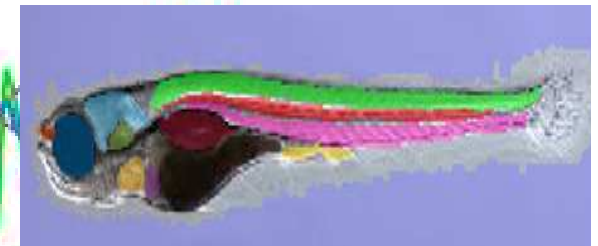


Object Detection

Semantic
Segmentation
Segment
Anything
Model (SAM)



Software Engineering and Data Science
SEIS 766: Vision A.I.
Dr. Chih Lai



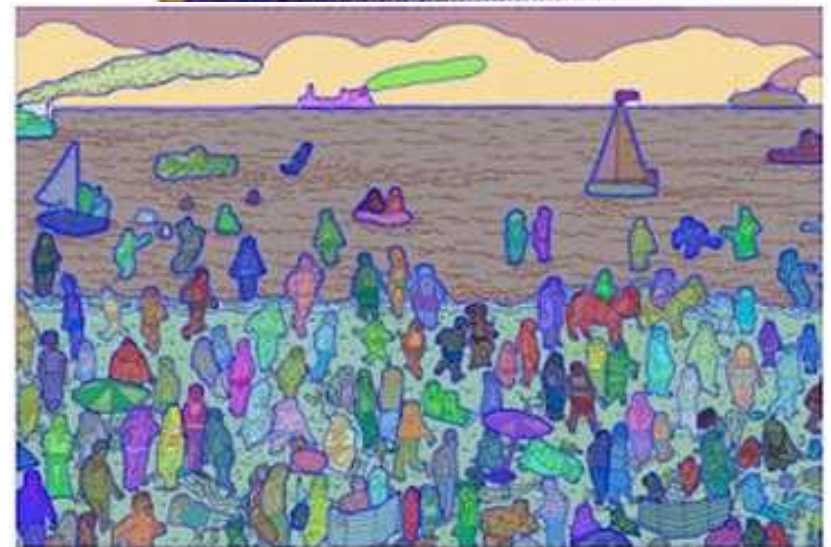
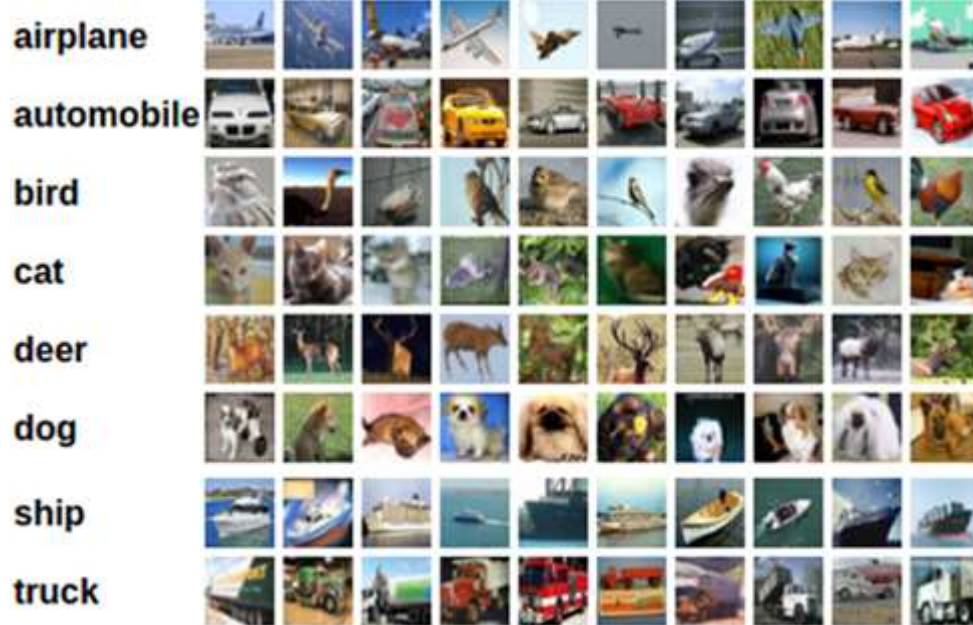
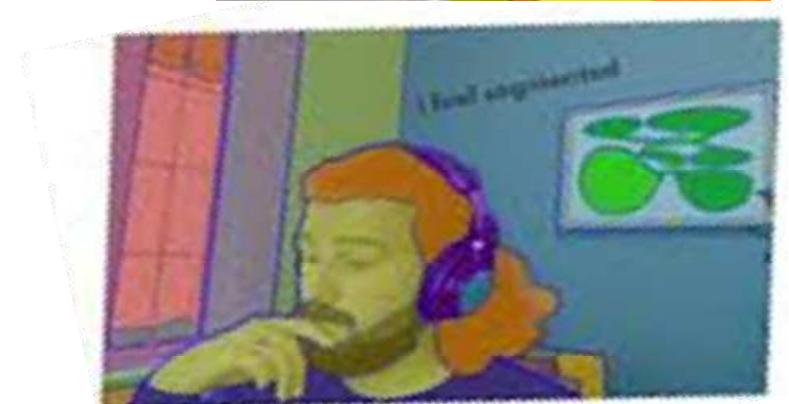
Outline

- Object Detection, Fast RCNN
- Object Detection, Semantic Segmentation, Labeling, Applications
- Encode + Decode CNN, Transposed Convolution, Pre-Trained CNN
 - SegNet, U-Net Family, Google's DeepLab V3+, Meta's SAM ✓
- Quality Evaluation of Object Detection
- Identifying Morphological Changes in Organs w/ Occlusion Map



Segment Anything Model / SAM

- By Meta, published in 2023
 - Train on 11 million images with 1 billion masks (labels)
 - More specifically, 11 million **NATURAL** images
- Is “*segment anything*” Truly Anything??
- Is it “*Semantic Segmentation*”??



SAM Demo 1

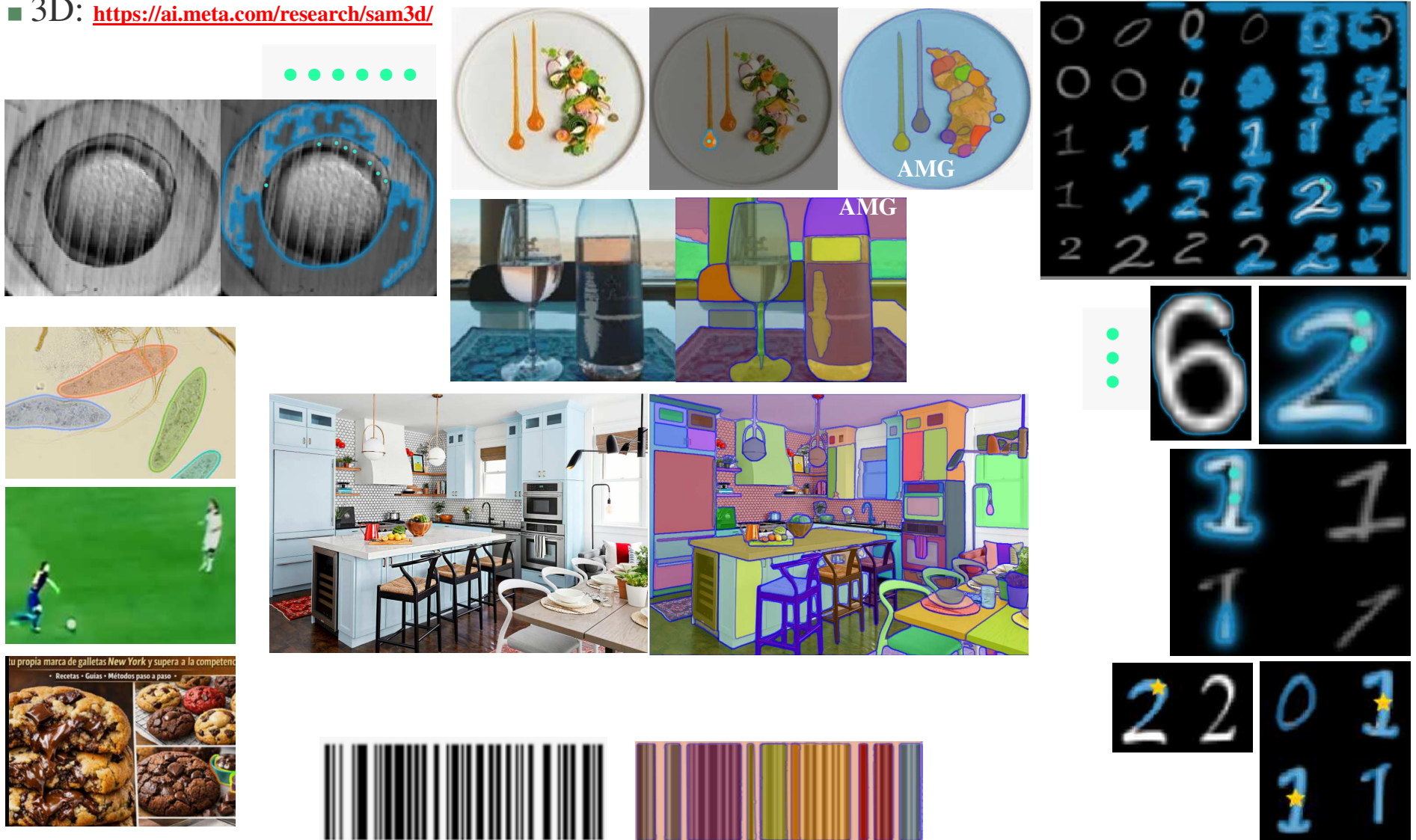
- Meta: <https://aidemos.meta.com/segment-anything/> <https://segment-anything.com/demo/>
- Hugging Face: <https://huggingface.co/spaces/Xenova/segment-anything-web>
- 3D: <https://ai.meta.com/research/sam3d/>

VAI_Slide_SAM_in_keras_hub.ipynb

VAI_Slide_SAM_ClickPrompt.m

VAI_Slide_SAM_AMG_Mode.m

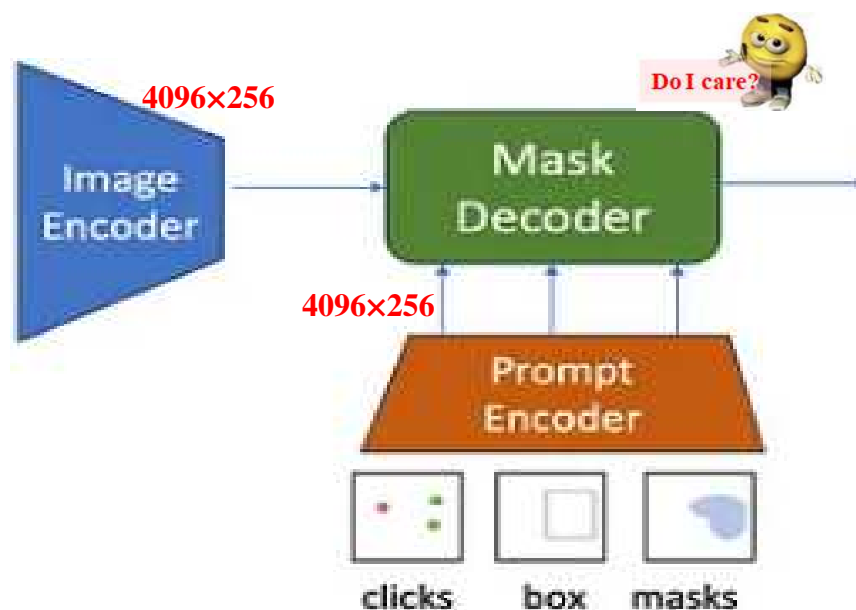
VAI_Slide_SAM_Medical.m



References & SAM Components

Is “*segment anything*” Truly Anything??

Is it “*Semantic Segmentation*”??



Example Code on Canvas:

- VAI_Slide_ClickPoints_In_Colab.ipynb
- VAI_Slide_SAM_AMG_DemoGitHub.ipynb
- VAI_Slide_SAM_in_keras_hub.ipynb

- VAI_Slide_SAM_AMG_Mode.m
- VAI_Slide_SAM_ClickPrompt.m
- VAI_Slide_SAM_Medical.m

(box prompt)

MATLAB:

<https://www.mathworks.com/help/images/ref/segmentanythingmodel.html>

<https://www.mathworks.com/help/images/interactively-segment-image-using-sam.html>

<https://www.mathworks.com/help/images/getting-started-with-segment-anything-model.html>

Keras: https://keras.io/keras_hub/api/models/sam/

SAMImageConverter

- [SAMImageConverter class](#)
- `from_preset` method

SAMBackbone model

- `SAMBackbone` class
- `from_preset` method

SAMImageSegmenter model

- `SAMImageSegmenter` class
- `from_preset` method
- `backbone` property
- `preprocessor` property

SAMImageSegmenterPreprocessor layer

- `SAMImageSegmenterPreprocessor` class
- `from_preset` method
- `image_converter` property

SAMPromptEncoder layer

- `SAMPromptEncoder` class

SAMMaskDecoder layer

- `SAMMaskDecoder` class

https://keras.io/keras_hub/guides/segment-anything-in-keras-hub/

SAM-1, Major Components

- Image encoder + Prompt encoder + Mask decoder
 - **Image Encoder** (ViT transformer) converts input images to *image embeddings* (Z_I)
 - **Prompt Encoders** uses MLP (dense) to convert *prompts* input (*clicks*) by users to Z_C
 - *Prompts* in SAM = *spatial* cue *Prompts* in LLM = *sequences* of words for *semantic* cue
 - **Mask Decoder** generates *masks* (segmentation results)



<https://learnopencv.com/segment-anything/>

Cross / Self Attention, Learn Where to Pay Attention



- Customer “I like something spicy” in $Q \rightarrow$ “spicy curry”, “mild soup”... on menu K
 - For each intent in Q , fetch the most relevant info in K , return converted Qv so they are preference-aware

- Text $Q \rightarrow$ Text K

- For each text in Q , fetch the most relevant info in K , return converted Qv so they are context-aware

The dog bit the man.	The cat chased the mouse.
The man bit the dog.	The mouse chased the cat.

- Text $Q \rightarrow$ Image K

- For each text in Q , fetch the most relevant info in K , return converted Qv so they are image-aware

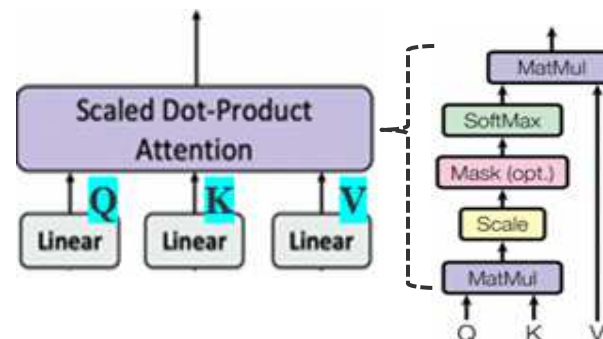


- Image $Q \rightarrow$ Text K

- For each image in Q , fetch the most relevant info in K , return converted Qv so they are text-aware

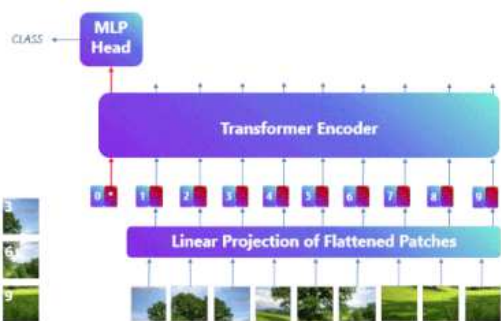
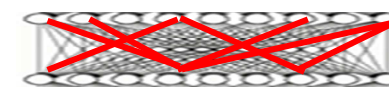
- Image patches $Q \rightarrow$ Image patches K

- For each patch in Q , fetch the most relevant info in K , return enriched Qv so they are content-aware



Vision Transformer (ViT)

Transform / **Attention**



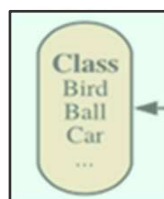
https://en.wikipedia.org/wiki/Vision_transformer



<https://arxiv.org/abs/2010.11929v2>

Image Embeddings = $64 \times 64 \times D$ (D FMs)

(D is embedding dim, last layer of XFomer)



For each patch, fetch the most relevant info in K, return enriched patches so they are *context-aware*

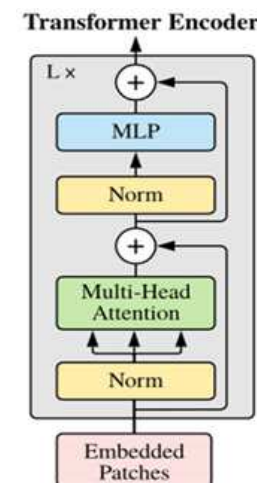
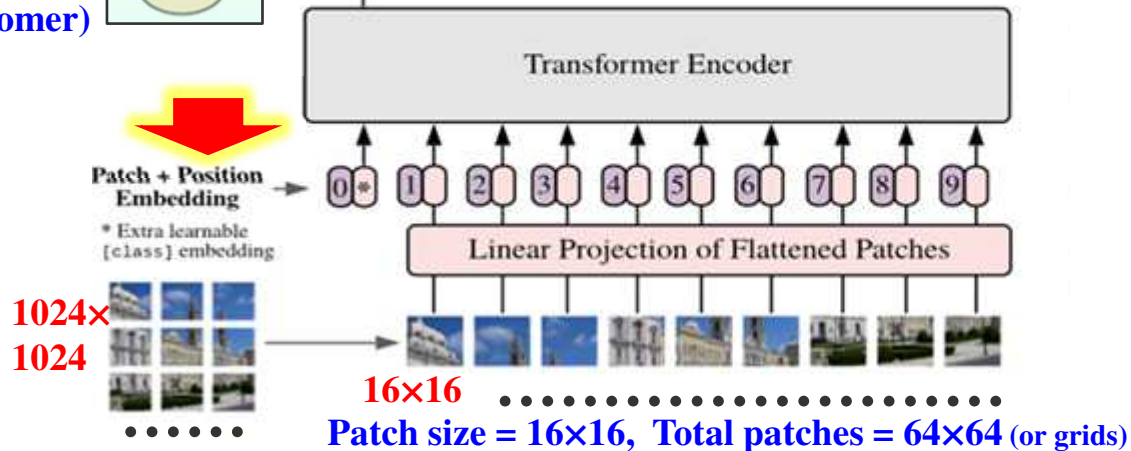
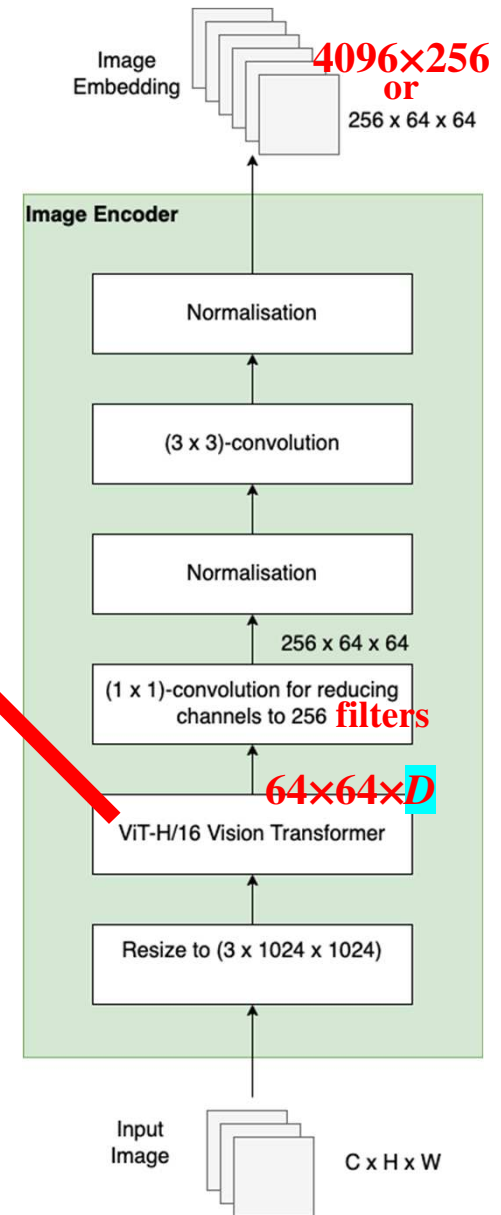
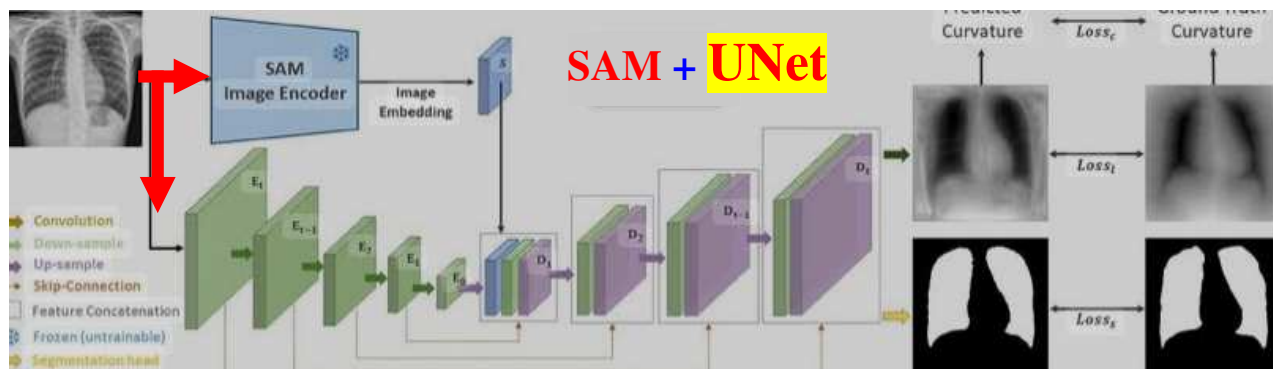
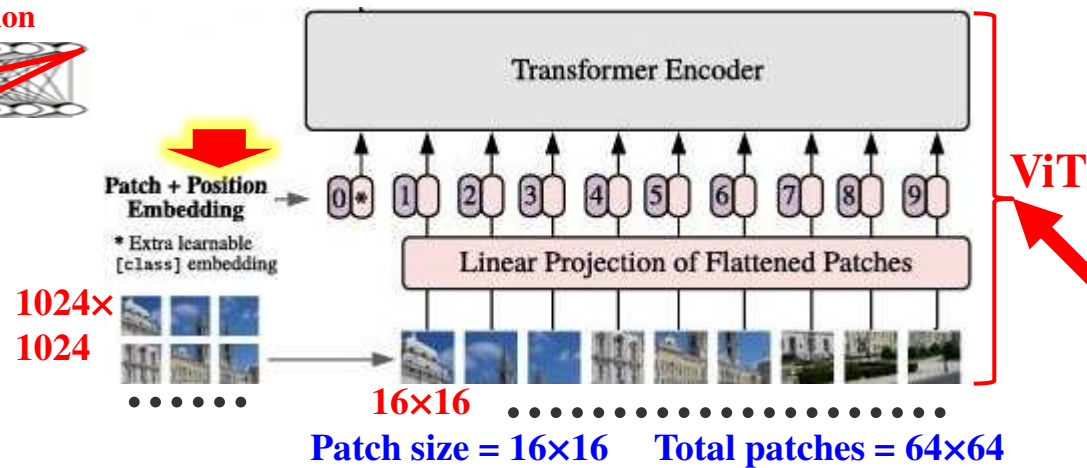
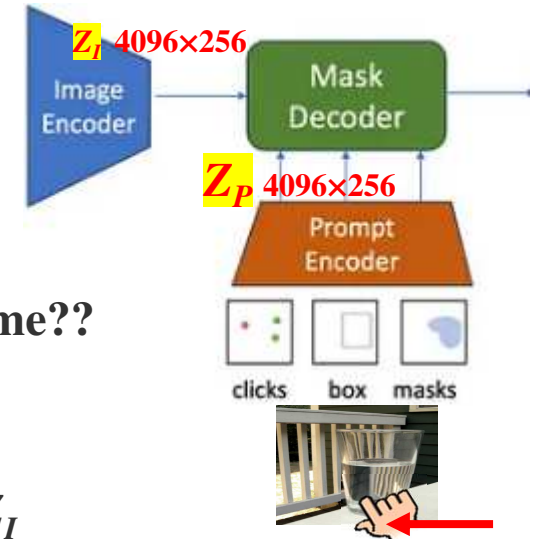


Image Encoder, More Detail

- Use **ViT** to convert images to embeddings Z
 - **ViT** (**V**ision **T**ransformer), purposes like convolution layers
- Segmentation may not precise enough in some domains...
 - Some work add UNet for detail info (better locations / boundaries)
 - Which domains???



Prompt Encoder



- Purposes of *prompts*, even just a click prompt
 - Ambiguity resolution, i.e. for multiple people / objects
 - Fine control granularity, interactive correction, but **cumbersome??**
- **Prompt encoder**
 - **Purpose:** align (x, y) coordinate into Z_P w/ image embeddings Z_I
 - Map $(x, y)_{2D}$ to a ***sinusoidal Coordinate encoding*** to capture **spatial info** at multi-frequency
 - Map $(x, y)_{2D}$ to ***Fourier position encoding*** so nearby pts are *different but smoothly related*
 - Then, map position encoding w/ small **MLP** to Z_P into same- D as image embeddings Z_I
 - So, Z_P & Z_I in *same space for transformer* in **Mask Decoder** can attend *patches near click*
 - Note, for all image patches, ViT produces a grid of high-dimensional embeddings

F.Y.I.

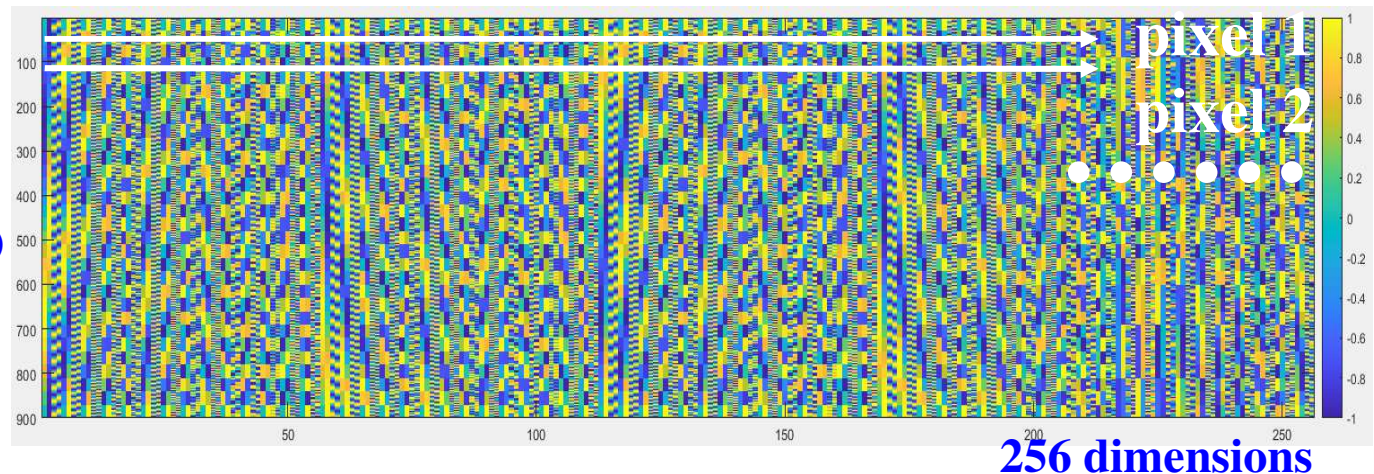
- 2-D ***sinusoidal Coordinate encoding*** differ from 1-D ***sinusoidal position encoding*** in LLM
 - $\gamma(x) = [\sin(2^0\pi x), \cos(2^0\pi x), \sin(2^1\pi x), \cos(2^1\pi x), \dots, \sin(2^{L-1}\pi x), \cos(2^{L-1}\pi x)]$, same as $\gamma(y)$
 - L = number of frequencies (e.g. 64)
 - So coordinate (x) expands to $2L$ values, together $(x, y) \rightarrow 4L$ values

2-D Coordinate Encoding in SAM vs. 1-D Position Encoding in LLM

- 2-D sinusoidal Coordinate encoding in SAM differ from 1-D sinusoidal position encoding in LLM

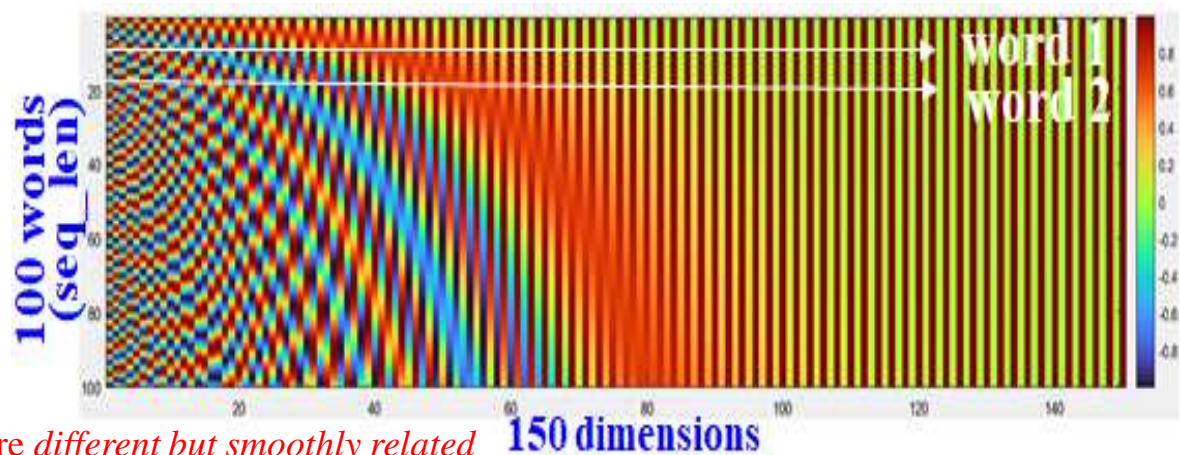
- $\gamma(\mathbf{x}) = [\sin(2^0\pi x), \cos(2^0\pi x), \sin(2^1\pi x), \cos(2^1\pi x), \dots, \sin(2^{L-1}\pi x), \cos(2^{L-1}\pi x)]$, same as $\gamma(\mathbf{y})$
- L = # of frequencies (e.g. 64), so coordinate (x) expands to $2L$, together (x, y) expand to $4L$
- For $30 \times 30 = 900$ nearby (x, y) , w/ frequency bands = 64, so each $(x, y) \rightarrow 4 \times 64 = 256\text{-D}$

$30 \times 30 = 900$
nearby (x, y)



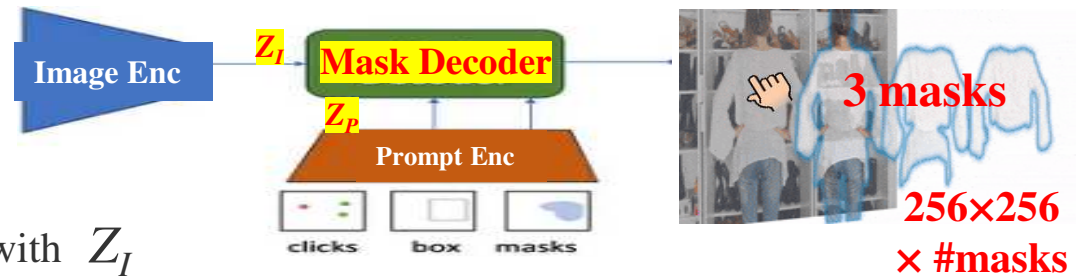
- 1-D sinusoidal position encoding in LLM

The dog bit the man.
The man bit the dog.
The cat chased the mouse.
The mouse chased the cat.
I am a teacher.
Am I a teach?



nearby words are different but smoothly related

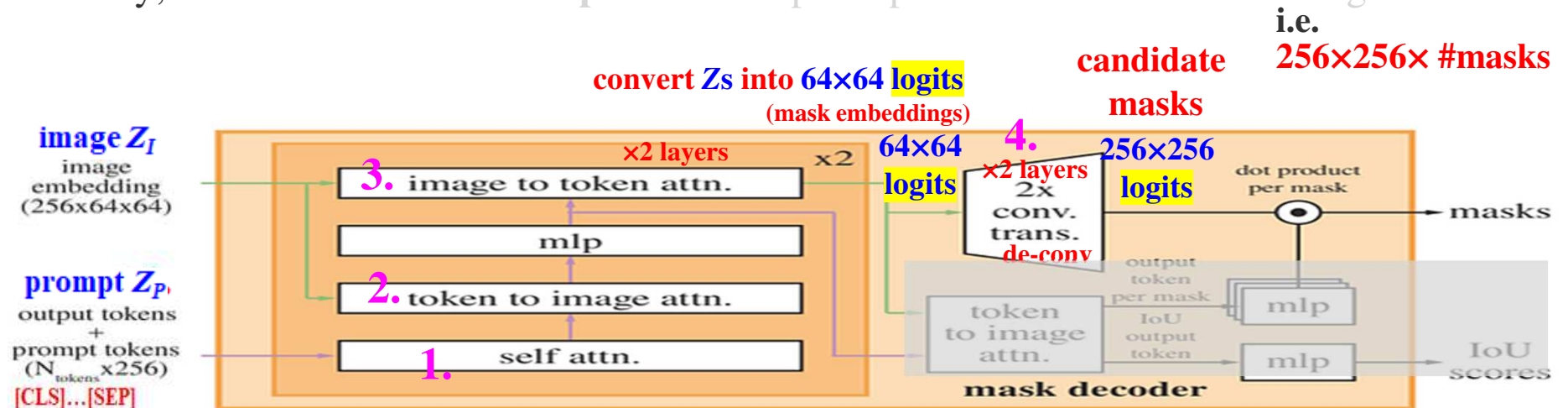
Light Weight Mask Decoder



■ *Mask Decoder* → link “clicks” Z_P with Z_I

- Produce multiple candidate masks **per prompt** (default, 3 masks per click)
- By convert Z_P & Z_I to predict logits ($\approx$ prob.) w/ 2x (attention & conv)
- **In short:** mask decoder combines Z_P & Z_I to classify into candidate masks
- Finally, **de-conv** + **bilinear interpolation** to up-sample to final size w/ smoothing

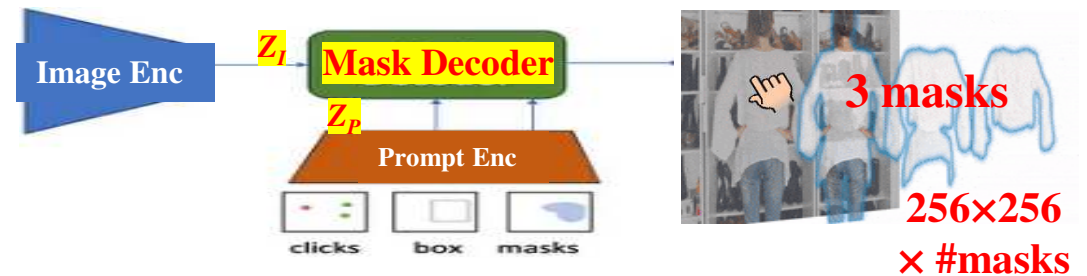
`outputs["masks"][0][0]`
[SAM_in_keras_hub.ipynb](https://github.com/Chih-Lai/SAM_in_keras_hub.ipynb)



<https://medium.com/@utkarsh135/segment-anything-model-sam-explained-2900743cb61e>

1. For each click prompt (Q), fetch the most relevant info in other prompts (K), returned converted prompts Q_v
2. For each prompt (Q / Q_v), fetch the most relevant info in images (K), return converted prompts (w/ image info)
3. For each image patch, fetch the most relevant info in prompts (K), return enriched image patches so they are *prompt-aware*
4. Put size back as input via de-conv

SAM Prediction with Point/Box Prompts



VAI_Slide_SAM_in_keras_hub.ipynb

load and create SAM through KerasHub

```
model = keras_hub.models.SAMImageSegmenter.from_preset("sam_huge_sa1b")
```

predict w/ box prompt... "masks" returned in "outputs" then combine w/ raw image

```
outputs = model.predict(
    {
        "images": image[np.newaxis, ...],
        "boxes": input_box[np.newaxis, np.newaxis, ...],
    }
)
```

access to "masks" in "output"

```
outputs["masks"][0][0]
```

1. Load the SAM model
2. Point prompts / Box prompts
3. Combine point + box prompts
4. Visualize masks



Mask Decoder + AMG

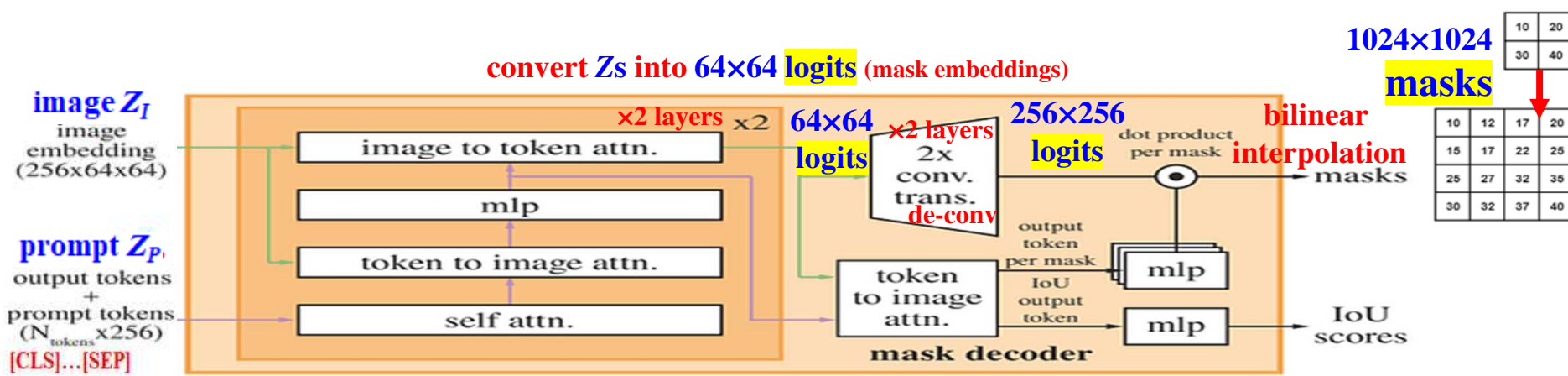


■ Mask Decoder

- Produce multiple candidate masks **per prompt** (default = 3) i.e. 3 masks per click
- By convert Z_P & Z_I to predict logits ($\approx$ prob.) w/ 2x (attention & conv) using MLP
- **In short:** mask decoder combines Z_P & Z_I to classify into candidate masks **i.e. $256 \times 256 \times \#masks$**
- Finally, **de-conv + bilinear interpolation** to up-sample to final size w/ smoothing

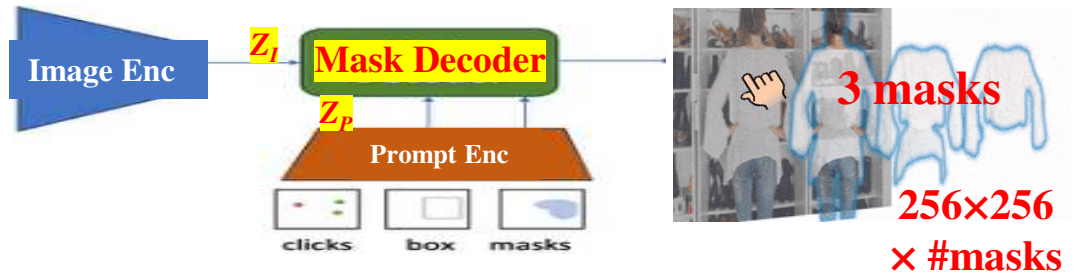
■ AMG mode (Automatic Mask Generation)

1. Generates **dense grid of “clicks”** in an image as *promptssssss*
 - i.e. sample an image w/ dense points & boxes, then mask decoder outputs masks for each sample
2. Post-processing steps (image process / *Non-Maximum Suppression*) **merge & filter overlapped masks**



<https://medium.com/@utkarsh135/segment-anything-model-sam-explained-2900743cb61e>

SAM Prediction with AMG Mode

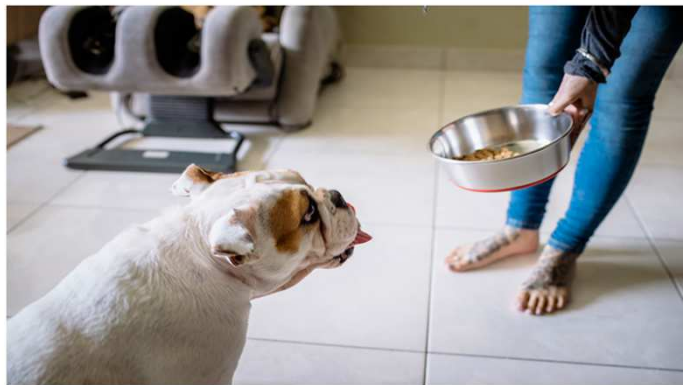


```
# via URL, load / create SAM model, then predictor inside
model = SegmentAnythingModel.from_preset("sam_huge_sa1b")
predictor = SAMPredictor(model)
```

[VAI_Slide_SAM_AMG_DemoGitHub.ipynb](#)

```
# further creating an AMG generator
mask_generator = SAMAutomaticMaskGenerator(predictor, ..... )
masks = mask_generator.generate(image, verbose=0)
```

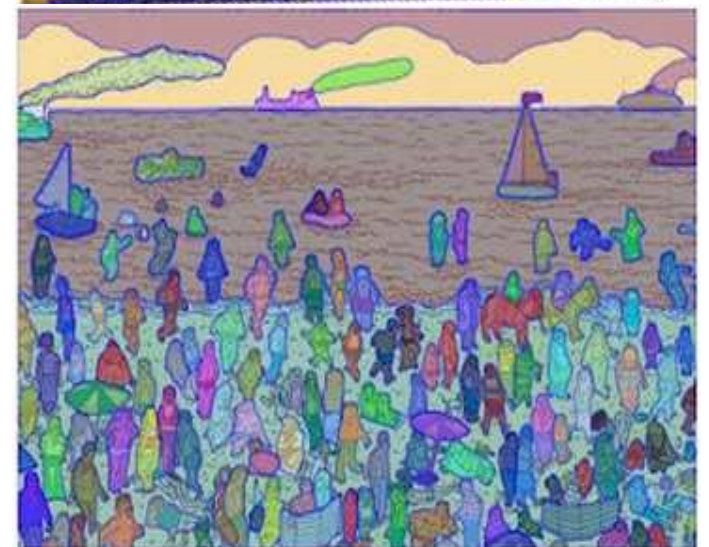
```
# Count total number of masks
num_masks = len(masks)
print(masks[0].keys()) # Access to masks
```



Segment Anything Model $\neq$ Semantic Segmentation

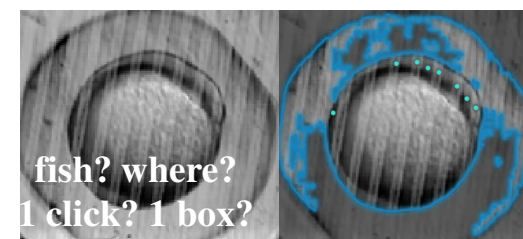
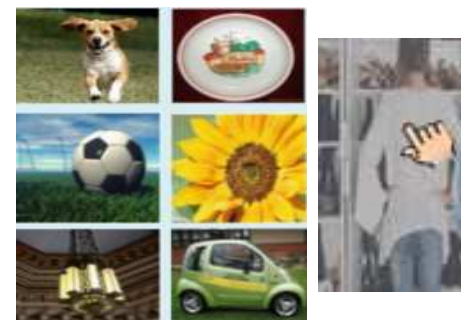


- **Semantic segmentation** focuses on “*semantics*”...
 - = label / classify object (e.g., *dog*, *chair*, *person*, *car*, *boat*)
 - = object class identification / recognition
- SAM is **NOT** trained w/ **explicit class labels**
 - It was trained to just delineates **regions**
 - Really good at: **region proposal / mask generation**
 - But it doesn't tell you *what* (i.e. *class*) the object is
- i.e. **NO semantic** segmentation in SAM
- Instead, a powerful **region segmentation** or **universal mask generator** based on **hidden Zs** learned via **DNN / Xformer over natural images**



“segment anything” ≠ Truly Anything

- SAM segments *anything* in *natural* photos
 - Objects in *natural* photos easy separable by texture/color/edges
 - One **click** (point prompt), sometime OK, often, not enough info
 - A prompt **box** cover too many mixed objects (tumor, bone, vessel, nerve...)
 - Too many clicks nullify *efficiency* advantage
- Struggle in domains, i.e. *medical images* (later slide)
 - Grayscale w/ unusual patterns, no clear boundary
 - **Overlap/blend** structures(vessels, nerves, bones), even experts disagree on boundaries
- How about industrial / manufacturing domain (i.e. for inspection)??
 - **Repetitive patterns** (i.e. *barcodes*) segment 1/1+ items to unrelated units
 - **Transparent or reflective objects**, like glass, water... for manufacture inspection
 - **Small defects** (scratches, cracks, soldering flaws, chips), examples **next slide**
 - **Extreme scales, adversarial / artistic images**
 - satellite images



SAM Struggle in Different Domains

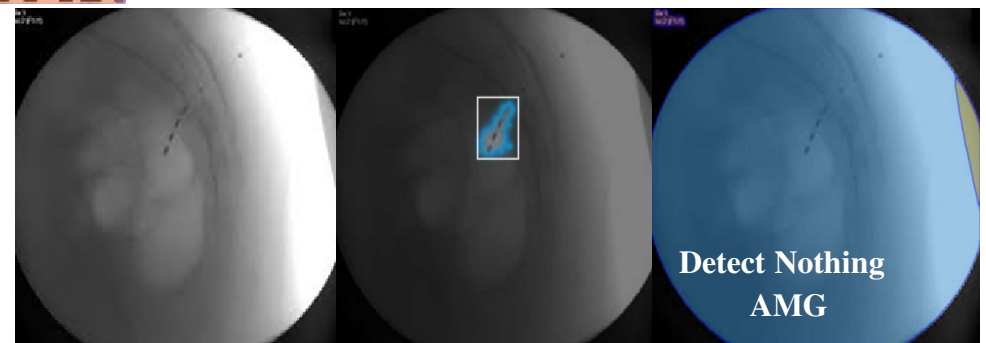
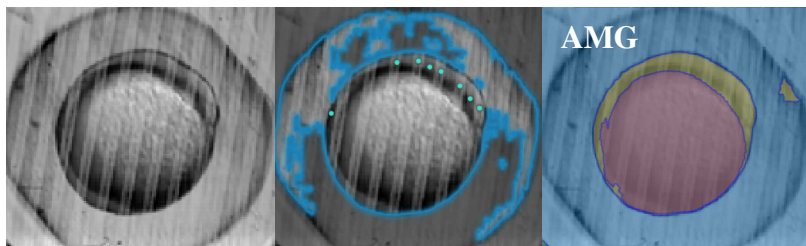
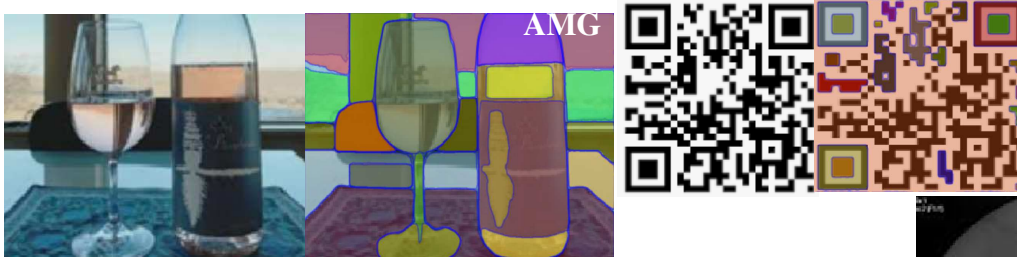
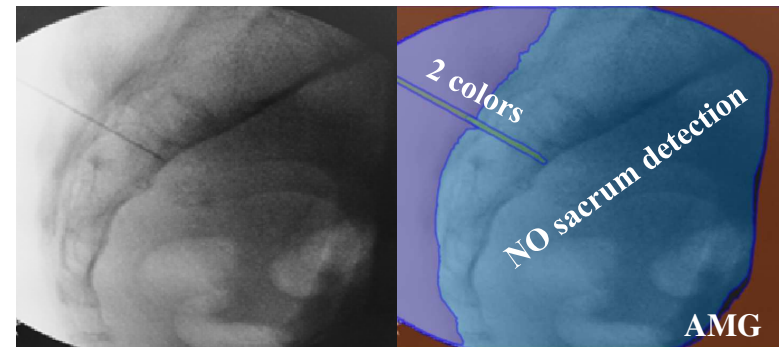
- Meta: <https://segment-anything.com/demo/>
- Huggin Face: <https://huggingface.co/spaces/Xenova/segment-anything-web>

VAI_Slide_SAM_in_keras_hub.ipynb
VAI_Slide_SAM_AMG_DemoGitHub.ipynb

VAI_Slide_SAM_ClickPrompt.m

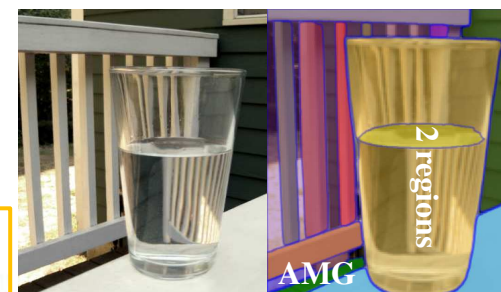
VAI_Slide_SAM_AMG_Mode.m

VAI_Slide_SAM_Medical.m



SAM Advantages + Disadvantage = Opportunities??

- Segment Anything *Model* $\neq$ Semantic Segmentation
- “*segment anything*” $\neq$ Truly Anything
- What opportunities do you see??



Example
Code
on Canvas:

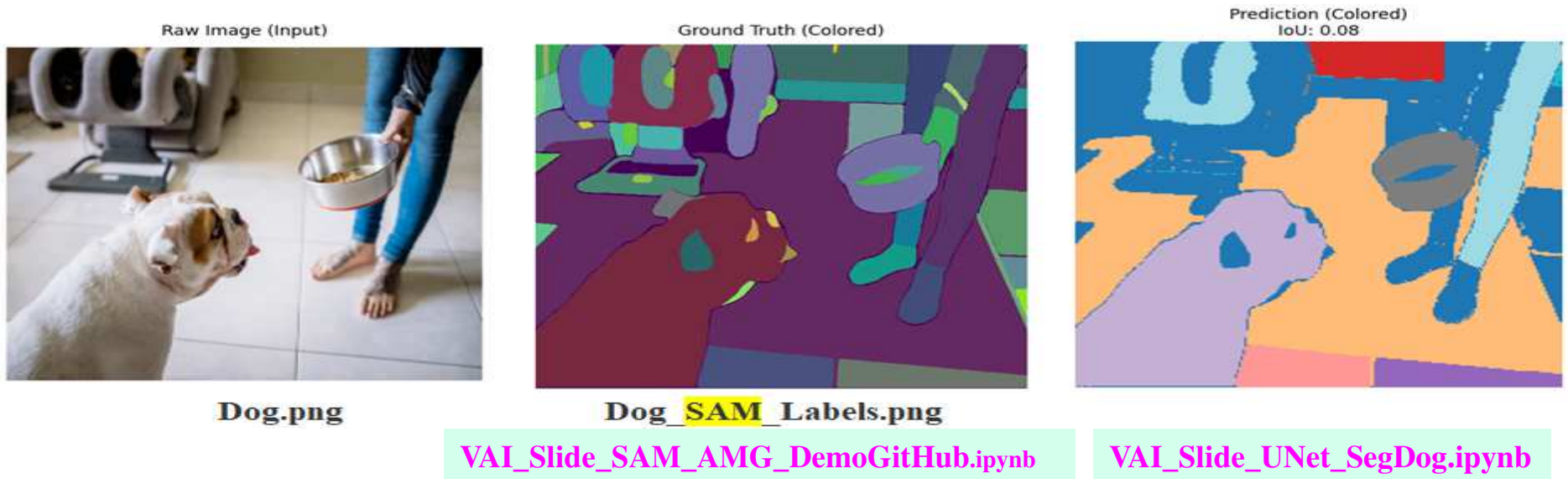
- VAI_Slide_ClickPoints_In_Colab.ipynb
- VAI_Slide_SAM_AMG_DemoGitHub.ipynb
- VAI_Slide_SAM_in_keras_hub.ipynb

- VAI_Slide_SAM_AMG_Mode.m
- VAI_Slide_SAM_ClickPrompt.m
- VAI_Slide_SAM_Medical.m



Using SAM for Auto- / Semi- Labeling **OR.....**

- **Overlap SAM on input image to increase info. for Semantic Segmentation**



Auto Mask Generation Using SAM

- For... semi-auto label annotation

766, 25F

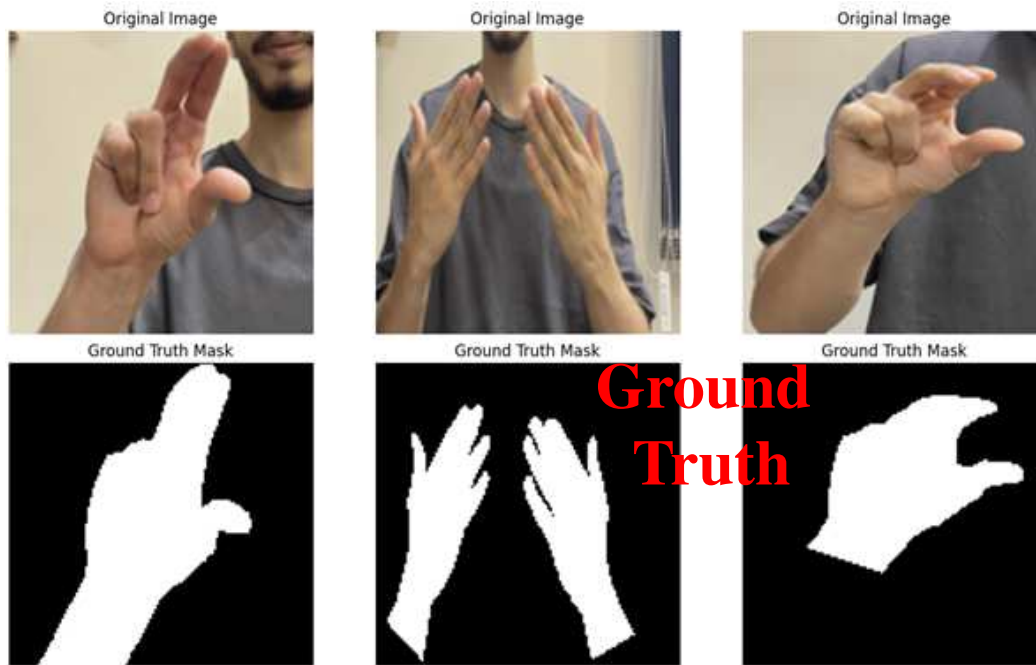
Exploring American sign language gestures

Max Helgestad,

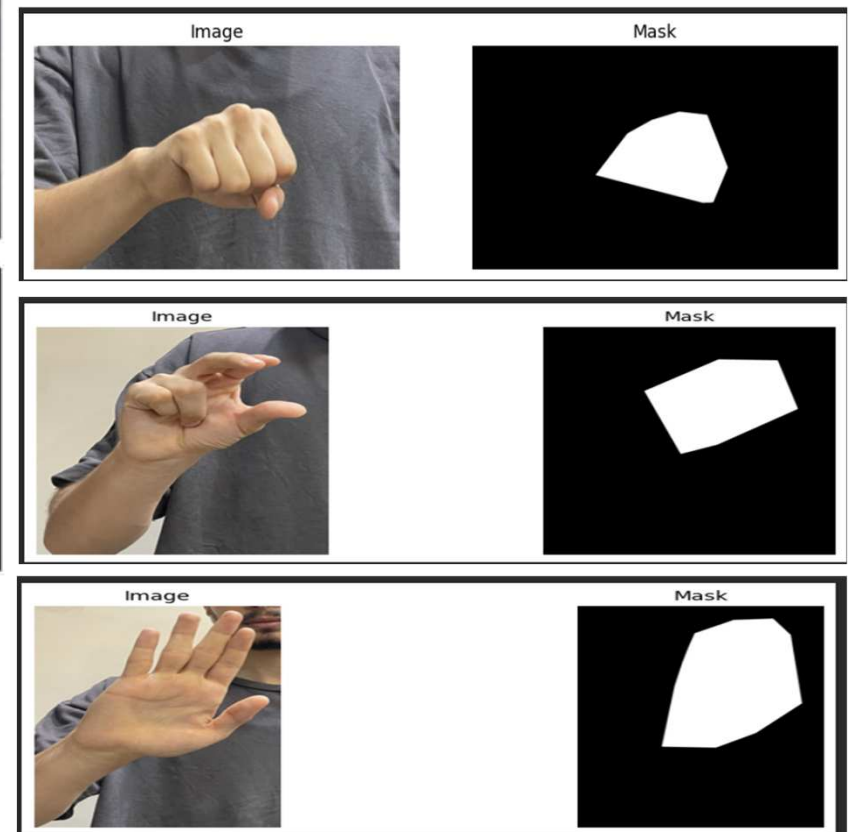
Brandon Vang,

Alicia John,

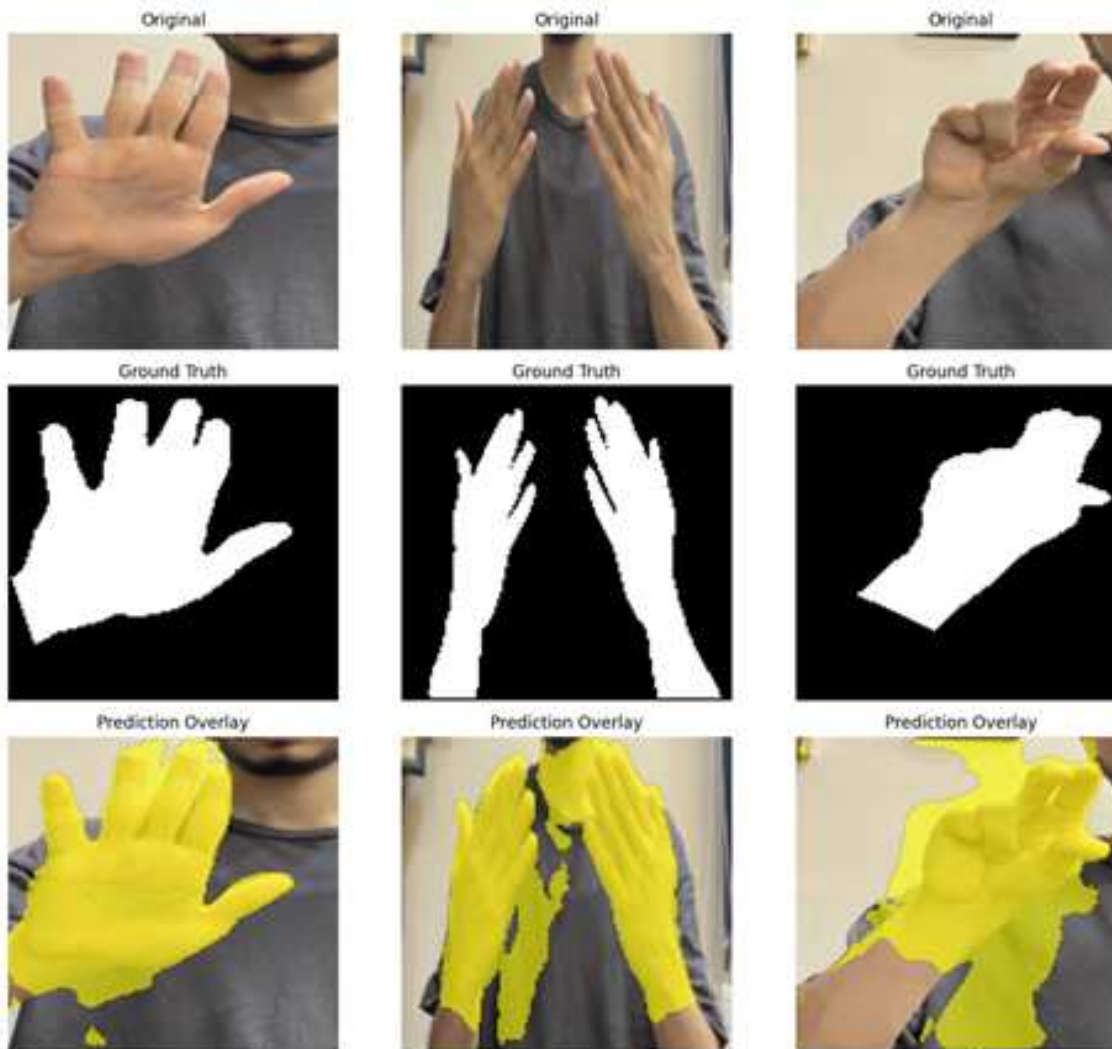
Mohammed Elshahawi



SAM Mask



Auto Mask Generation Using SAM→ THEN to U-Net for Semantic Segmentation

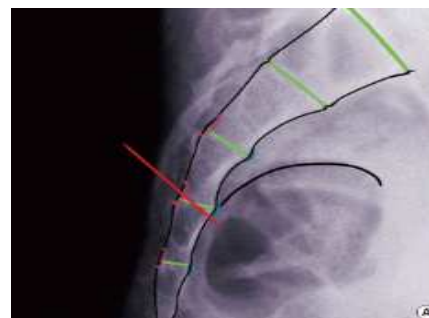
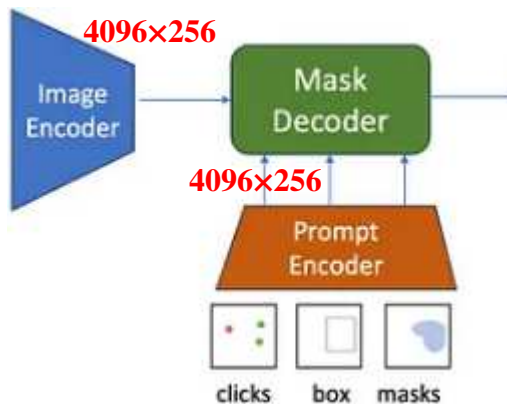
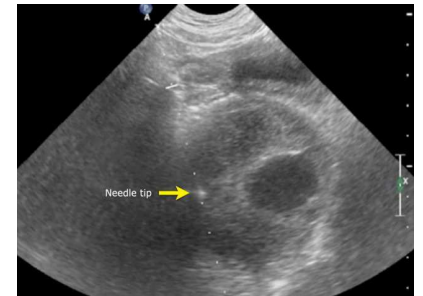


766, 25F
Exploring American
sign language gestures
Max Helgestad,
Brandon Vang,
Alicia John,
Mohammed Elshahawi

Why SAM Struggle in Medical Images?

■ Struggle in structurally different domains, i.e. medical images (next slide)

- **Grayscale, unusual patterns no clear boundary**
- Even experts disagree on boundaries
- OK finding *clear* regions (like a whole lung)
- But struggles with *fine structures* (tiny lesions, blood vessels, tumors)
- Many structures *overlap or blend* (blood vessels, nerves, bones)



■ **MedSAM 2024**

- Use massive medical images w/ focus on anatomical structures & pathologies (i.e. cancer)
- Fine-tune **mask decoder** while keep everything else

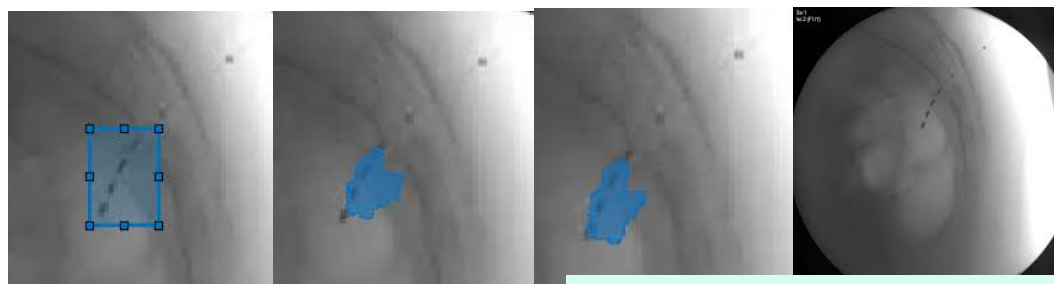
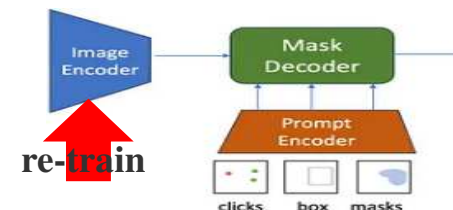
MedSAM 2024 in Matlab, Re-Train Image Encoder w/ Medical Images

■ *medicalSegmentAnythingModel*, <https://www.mathworks.com/help/medical-imaging/ref/medicalsegmentanythingmodel.html>

● *segmentAnythingModel*, <https://www.mathworks.com/help/images/ref/segmentanythingmodel.html>

● *imsegsam*, <https://www.mathworks.com/help/images/ref/imsegsam.html>

● <https://github.com/bowang-lab/MedSAM>



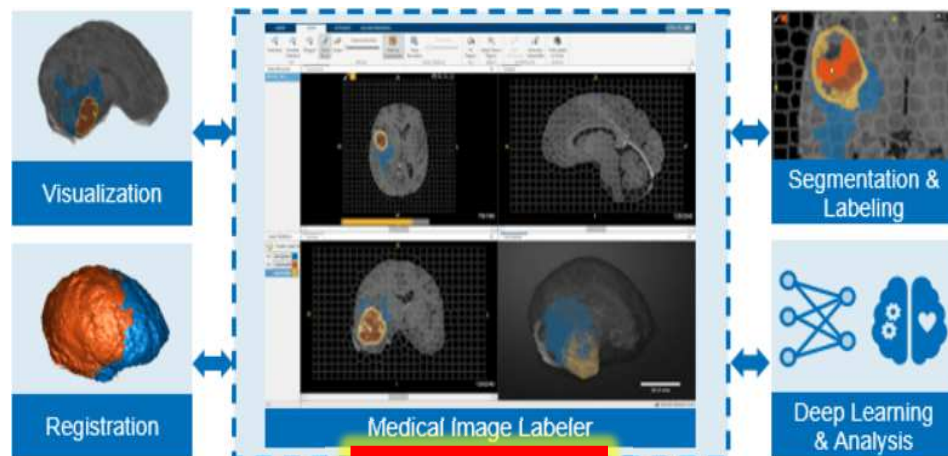
VAI_Slide_SAM_Medical.m

■ Matlab Medical Imaging Toolbox, visualize, *register*, *segment*, label 2D & 3D medical images

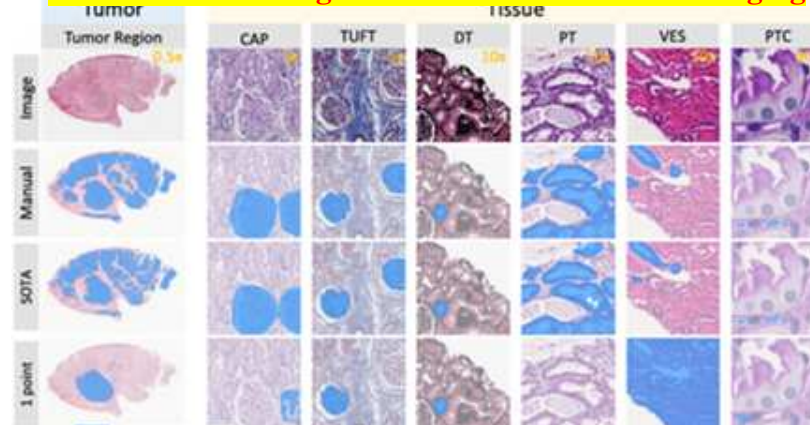
● <https://www.mathworks.com/products/medical-imaging.html> **Video**

● <https://www.mathworks.com/help/medical-imaging/>

● <https://www.mathworks.com/help/medical-imaging/ug/get-started-with-medical-image-labeler.html>

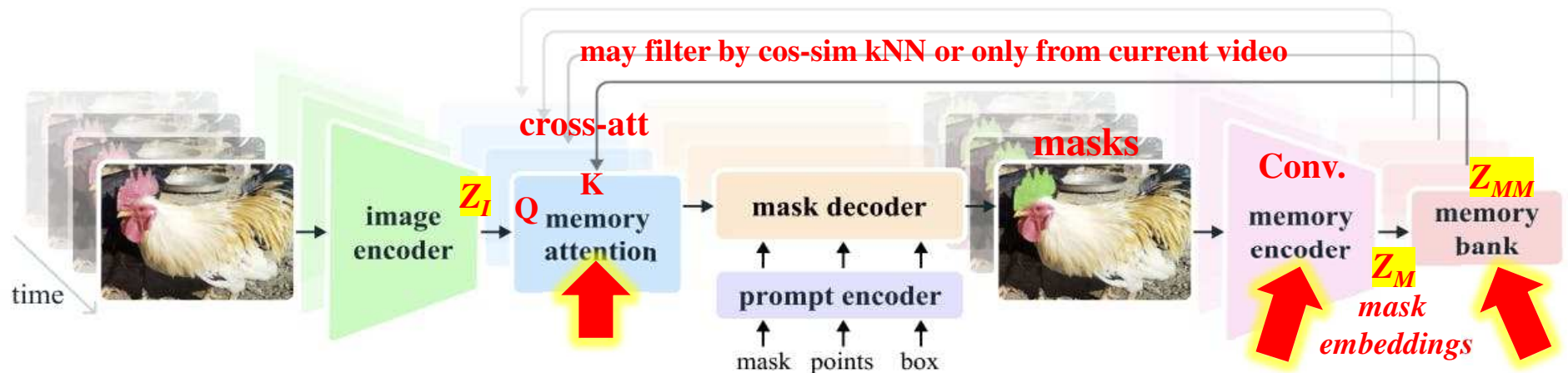


**Segment Anything Model (SAM) for Digital Pathology:
Assess Zero-shot Segmentation on Whole Slide Imaging**



SAM-2, Memory Decoder & Memory Bank <https://ai.meta.com/sam2/#demo>

- SAM-1 captures **regions + shape + texture cues** in images, why not store them??
- SAM-2 is a unified model for **both images & videos** & **few-shot learning**
 - **Memory encoder** convert **masks** as *mask embeddings* Z_M & save in **memory bank** as Z_{MM}
 - Later retrieve from **bank** to guide segmentation on new-domain images (or videos) **w/o** full retrain
 - **Cross-att** use few domain example Z_{MM} in bank to align new image Z_I for better segment
 - **Also** Z_{MM} in bank **anchors** to previously learned clusters, prevent future drift & stability
- It can enable few-shot or 0-shot adaption in new domains, example:
 - i.e. a new liver tumor mask from a new image → compare its Z_M to stored “tumor-like” Z_{MM}
 - Helps refine the segmentation, even if SAM’s raw logits are noisy



Comparison, All to Preserve Image Detail for Precise Object Locations

- They are for different purposes, not just a linear upgrade over years

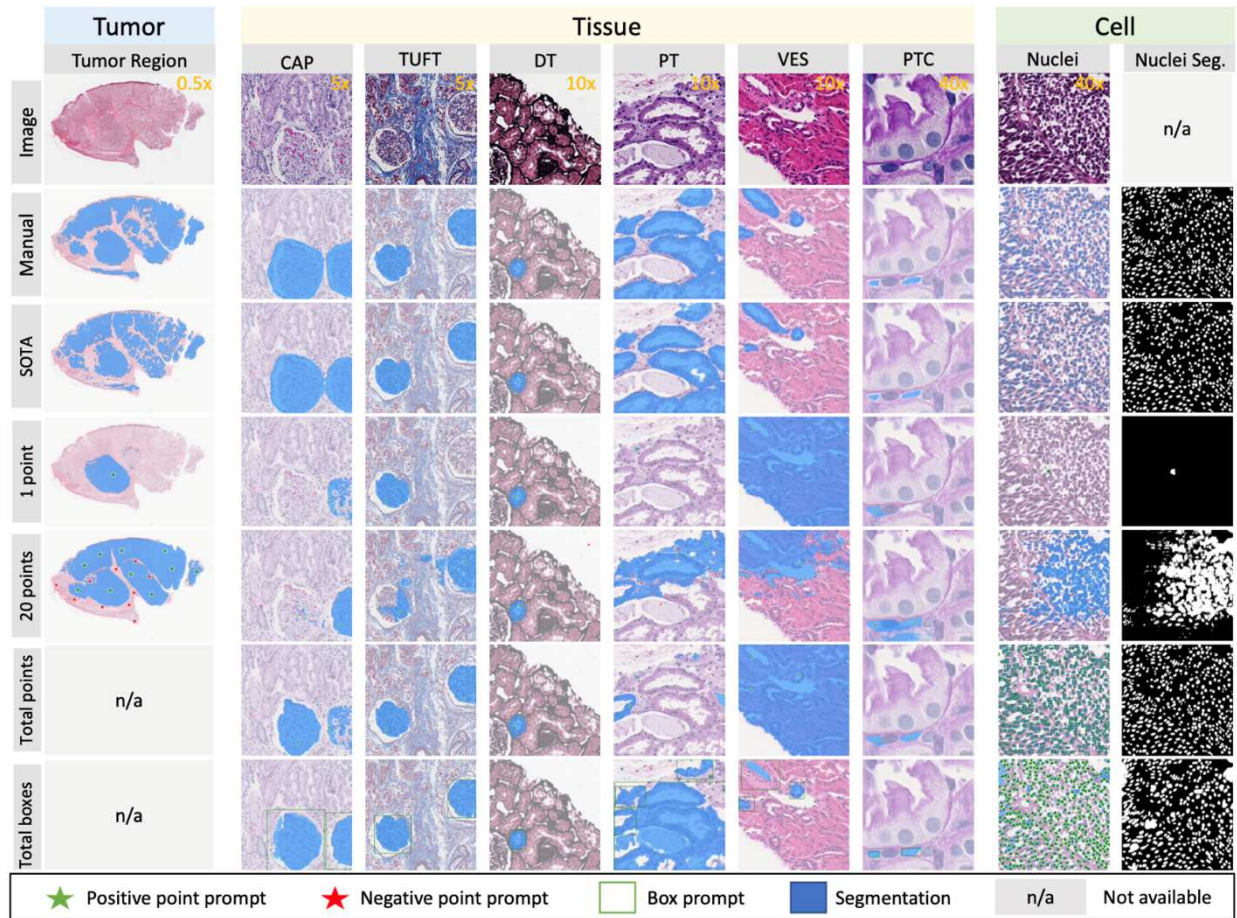
Feature	DeepLabV3+ (2018)	U-Net3+ (2015 / 2019 / 2020)	SAM / Segment Anything (2023)
Architecture Type	CNN-based, with dilated convolutions / Atrous Convolutions	CNN encoder-decoder with full-scale skip connections (dense connections across all scales)	Transformer-based ViT with multiple encoder + lightweight mask decoder
Generalization	Moderate → needs retraining/fine-tuning for new domains	Moderate → needs retraining/fine-tuning for new domains	Very high → zero-shot or prompt-based segmentation (but = regions)
Prompting	No user prompt required (other than labels)	No user prompt required (other than labels)	Supports user prompts (points, boxes) for interactive region segmentation
Parameter Size	~40–60M (depending on backbone)	~25–40M	Very large: ~600M (ViT-H encoder) Trained on 11M images, 1 billion labels, high resolution images
Strengths	Strong semantic segmentation; robust to scale variation (via ASPP); good for natural images Faster for real-time surgery w/upgrade	Excellent multi-scale fusion; strong performance on biomedical & fine-structure segmentation	Foundation model; highly generalizable for regions ; interactive segmentation; strong zero-shot ability
Weaknesses	Rough boundary precision; No instance separation	Heavy computing ; slower due to full-scale skip , No instance separation	Very large model, memory intensive ; domain gap on medical images without adaptation, No instance separation

Appendix

- Segment Anything Model (SAM) for Digital Pathology: Assess Zero-shot Segmentation on Whole Slide Imaging

Segment Anything Model (SAM) for Digital Pathology: Assess Zero-shot Segmentation on Whole Slide Imaging

■ <https://www.vanderbilt.edu/valiant/2025/04/23/segment-anything-model-sam-for-digital-pathology-assess-zero-shot-segmentation-on-whole-slide-imaging/>



F.Y.I.

Figure 1. Qualitative segmentation results. The SOTA methods are compared with SAM method with different prompt strategies.